192425037

EXP24:

```
#include <stdio.h>

long long egcd(long long a, long long b, long long *x, long long *y) {
    long long x1, y1, g;
    if (b == 0) { *x = 1; *y = 0; return a; }
    g = egcd(b, a % b, &x1, &y1);
    *x = y1;
    *y = x1 - (a / b) * y1;
    return g;
}

int main() {
    long long e, n, p = 0, q, phi, x, y, d, i;

    printf("Enter public key (e n): ");
    scanf("%lld %lld", &e, &n);

    /* trial and error to find p and q */
    for (i = 2; i * i <= n; i++)
        if (n % i == 0) { p = i; break; }
    q = n / p;
    phi = (p - 1) * (q - 1);
    printf("p = %lld, q = %lld\n", p, q);
    printf("phi(n) = (p-1)(q-1) = %lld\n", phi);

    /* extended Euclid: e*x + phi*y = 1  =>  d = x mod phi */
    egcd(e, phi, &x, &y);
    d = ((x % phi) + phi) % phi;
```

```
    printf("Multiplicative inverse of %lld mod %lld: d = %lld\n", e, phi, d);
    printf("Check: (e*d) mod phi = %lld\n", (e * d) % phi);
    printf("Private key = {%lld, %lld}\n", d, n);
    return 0;
}
```

OUTPUT:

```
Enter public key (e n): 31 3599
p = 59, q = 61
phi(n) = (p-1)(q-1) = 3480
Multiplicative inverse of 31 mod 3480: d = 3031
Check: (e*d) mod phi = 1
Private key = {3031, 3599}
```